

Test-Driven Development

Gregory S. DeLozier, PhD

A requirement such as “convert a string into a number” leaves quite a bit undecided. Does the string contain one digit or several? Can it have a decimal point? What should happen when it contains a letter?

Designing a test makes us decide something specific. Given "7", we expect 7.0. We can write that expectation as code before writing the feature. Now we have a small, concrete specification that the computer can check.

This is why we introduce testing early. Creating tests is part of creating specifications. We define one small behavior, implement it, and then specify the next behavior. We do not need a complete specification for every possible number before we begin. We do need to know what the next piece is supposed to do.

Test-driven development, or **TDD**, gives us a way to work in that order: write a test, see it fail, implement the behavior, and improve the working code (1). In this chapter, we will use ordinary Python test procedures called from `main`. Our example converts strings containing ASCII digits into floating-point values, starting with a single digit and adding features one at a time.

Testing gives us evidence

The purpose of testing is to collect evidence of something of value to someone. Here, we want evidence that our converter produces the values its callers expect. A test supplies an input, calls the converter, and compares the actual result with the expected result.

Printing a result is a start, but it still leaves us to decide whether it is right. A test should make that comparison itself. Otherwise, after a few changes, we may stop looking carefully at the output.

A passing test tells us about the case we checked. It does not establish that every possible input works. A converter that returns 7.0 for every input passes a test for "7". It will not survive a test for "3". That is a useful limit to discover early.

A testing agency checks addition

Before building the converter, consider a different job. We are a testing agency. Someone supplies a function called `add(x, y)` and asks us to check that it adds two numbers. We can call it, but for now we cannot inspect its implementation. That is **black-box testing**: our evidence comes from inputs and observable results.

Our initial scope is ordinary, finite numbers. We expect their sum, including when the operands differ or are negative. We will use small integers and simple fractions so that rounding does not distract us from the exercise.

The first test

The supplied functions live in `code/math.py`. In a separate file, import the first version under the name `add`:

```
from math import add1 as add

def test_two_plus_two():
    assert add(2, 2) == 4

def main():
    test_two_plus_two()

if __name__ == "__main__":
    main()
```

Run the file with Python, without `-0`, so the assertions execute. This test passes. We have evidence that one call gives the expected result. What else should we try?

Add a test for `add(3, 3) == 6` and call it from `main`. It fails. The supplier provides `add2`; change the import to `from math import add2 as add`. Both tests now pass. Keep them and add `add(4, 4) == 8`. That fails too.

We keep repeating this process. Each replacement passes the tests that exposed its predecessor, but a different kind of input exposes another problem.

New case	Expected result	Version it exposes	Next version passes the accumulated cases
<code>add(2, 2)</code>	4	None yet	All five
<code>add(3, 3)</code>	6	<code>add1</code>	<code>add2</code>
<code>add(4, 4)</code>	8	<code>add2</code>	<code>add3</code>
<code>add(2, 3)</code>	5	<code>add3</code>	<code>add4</code>
<code>add(-2, -3)</code>	-5	<code>add4</code>	<code>add5</code>

The unequal-operands case matters more than simply trying another equal pair. Negative inputs introduce another distinction. We are looking for kinds of behavior we have not examined, rather than collecting more examples of the same thing.

Keep the earlier evidence

The growing suite uses named test procedures:

```
def test_two_plus_two():
    assert add(2, 2) == 4

def test_another_equal_pair():
    assert add(3, 3) == 6

def test_larger_equal_pair():
    assert add(4, 4) == 8

def test_unequal_positive_values():
    assert add(2, 3) == 5
    assert add(3, 2) == 5

def test_negative_values():
    assert add(-2, -3) == -5
    assert add(-2, 3) == 1
    assert add(2, -3) == -1

def test_zero_and_fractions():
    assert add(0, 0) == 0
    assert add(0, 7) == 7
    assert add(7, 0) == 7
    assert add(0.25, 0.5) == 0.75
    assert add(-0.25, 0.5) == 0.25
```

Call each procedure from main. The complete file, `code/test_maths.py`, includes those calls. It starts with `add1`, so running the complete suite intentionally fails at the second test. During the exercise, begin with the first call and add the others as each new question arises. Change only the imported version when the supplier provides a replacement.

The quarter and half values here are exactly representable in binary floating point. We will deal with approximate decimal comparisons in the converter example.

With `add5` imported, every test above passes. The suite checks several plausible sources of error. It is still small, but it contains more useful evidence than hundreds of positive equal-operand examples alone. More extensive testing could add larger magnitudes or other boundary cases appropriate to the agreed input range.

After the exercise: inspect the implementations

Here is what the supplier actually gave us:

```
def add1(x, y):  
    return 4.0  
  
def add2(x, y):  
    if x == 2:  
        return 4  
    if x == 3:  
        return 6  
  
def add3(x, y):  
    return x * 2  
  
def add4(x, y):  
    return abs(x) + abs(y)  
  
def add5(x, y):  
    if x == 123.456:  
        return 8675309  
    return x + y
```

The first version always gives the same answer. The second recognizes two values of x . The third doubles x and ignores y . The fourth adds magnitudes and loses the signs. Our tests exposed each of these limitations.

The fifth behaves differently. It performs addition except for one deliberately chosen value of x . Once we see the code, it is easy to write a failing test:

```
def test_revealed_exception():  
    assert add(123.456, 0) == 123.456
```

With `add5`, the actual result is 8675309. Using zero for the second operand keeps the expected result clear and avoids introducing an additional rounding question.

Would black-box testing find that last error?

How many test cases would we have to write? There is no useful fixed answer without a strategy for selecting inputs. A test that happens to use 123.456 finds it immediately. A suite with millions of other values of x misses it. Random testing might discover it, but random does not mean every possible value will eventually be tried during a practical test run.

Strictly exhaustive testing of the agreed input domain would include the exceptional value. The difficulty is doing it. Even when the machine representation makes the domain finite, testing every pair of values can be far beyond our available time. We should not confuse “a lot of tests” with “every possible input.”

Code inspection supplies a different kind of evidence. The exceptional branch is visible, and we can ask what requirement justifies it. For this addition function, none does. Inspection and execution complement each other; neither needs to carry the entire argument for correctness alone.

The evidence is still worth having

Does failing to discover that last exception make the passing suite worthless? No. Ask how often we encounter a deliberately hidden exception like this in an ordinary addition routine. It is a contrived example that establishes a limit on what we can conclude. It does not erase the useful distinctions our earlier tests examined.

The test for 123.456 is decisive evidence that this implementation is wrong. Once we know about the defect, keeping that test also guards against its return. Before seeing the implementation, though, we had no particular reason to favor that number over millions of others. The targeted test and the general suite answer different questions.

We can reasonably trust a comprehensive set of tests covering plausible mistakes without claiming certainty. The required confidence depends on what happens if the software is wrong. That confidence becomes better grounded when tests are combined with inspection and a clear understanding of the requirements.

The two lessons belong together: practical testing cannot exercise everything, so inspect code too; and useful testing remains useful even when it cannot rule out every imaginable defect.

In the agency exercise, we checked software someone else supplied. Now we will bring that way of thinking into development itself: decide on an expected result before writing the behavior that produces it.

Tests as concrete specifications

A useful test contains a decision. The input "007" should produce 7.0, so leading zeros are allowed. The input "1.2.3" should raise an error, so a second decimal point is forbidden. We have made parts of the requirement concrete before implementing them.

The expected result must come from the intended behavior. Running the current code and recording whatever it returns would preserve its behavior, including mistakes, without establishing that it does what we need.

Examples alone do not describe every case. Several examples can express a broader rule, and a short sentence can state that rule directly. "Accept any single ASCII digit" is clearer than leaving someone to infer the whole requirement from a test for "7". The rule and its tests support each other.

This also gives the team something specific to discuss. Should ".5" be accepted? We can settle that question and record the answer in a test. If our understanding changes, we revise the specification deliberately and then change the implementation.

A small cycle

The usual name for the cycle is **red, green, refactor**. Red means a test fails. Green means the tests pass. Refactoring improves the structure of working code while preserving its behavior (1). Wikipedia also provides an overview of TDD and its terminology (2).

Step	What we do	Evidence we want
Red	Specify one behavior in a test and run it	It fails because that behavior is missing
Green	Add enough code to support that behavior	The new test and all earlier tests pass
Refactor	Improve the working code	The same tests still pass

These are small steps. We do not write every test before beginning the implementation. Keep a short list of examples we might need, choose one, and revise the list as we learn (1).

There is no test framework in this first example. Ordinary functions called from `main` are enough to make the cycle visible.

The converter's first requirement

American Standard Code for Information Interchange (**ASCII**) assigns character codes to letters, digits, and other characters. We will accept the digit characters "0" through "9". The input "7" is text; the output `7.0` is a number we can use in arithmetic.

We turn the general conversion requirement into our first small specification:

Given the string "7", return the floating-point number 7.0.

Put this program in a file named `converter.py`:

```
def parse(text):
    return 0.0

def main():
    actual = parse("7")
    if actual != 7.0:
        raise AssertionError(f"Expected 7.0, got {actual!r}")
    print("All tests passed.")

if __name__ == "__main__":
    main()
```

Run it with `python3 converter.py`. It raises an `AssertionError`: we expected `7.0`, but received `0.0`. The program also exits with a failure status. That matters because another program can eventually run these tests and notice a failure without reading the screen.

This is the failure we wanted. A misspelled function name or a syntax error would also stop the program, but neither would tell us whether the conversion behavior was missing. Read the failure before changing the converter.

A plain Python `assert` can also express a test, as in `assert parse("7") == 7.0`. Run such tests without Python's `-O` option, which removes assertions. The explicit checks here keep running even when that option is used.

Now replace the converter with:

```
def parse(text):  
    return 7.0
```

Run the test again. It passes.

This implementation is temporary. Returning a constant is a way to get the first test working, not a rule that every TDD exercise must begin with a constant. We have established that the test calls the function and detects a wrong answer. The next example will require more useful behavior.

Any single digit

Keep the first test and add a test for "3". The constant implementation returns `7.0`, so the new test fails. We now need to distinguish digits.

For ASCII digits, subtracting the character code for "0" gives the digit's numeric value. Python's `ord` returns a character's code point; these ten digits have consecutive values. The call to `float` below converts that small integer result to a floating-point value. It does not parse the input string.

```
def parse(text):  
    return float(ord(text[0]) - ord("0"))
```

Run both tests. Then add "0" and "9" as boundary examples. Those tests already pass. That is useful additional evidence; it does not require inventing another implementation change.

As the checks repeat, pull their common work into a small test procedure:

```
def check(text, expected):  
    actual = parse(text)  
    if actual != expected:  
        raise AssertionError(  
            f"{text!r}: expected {expected!r}, got {actual!r}")  
  
def main():  
    check("7", 7.0)  
    check("3", 3.0)  
    check("0", 0.0)  
    check("9", 9.0)  
    print("All tests passed.")
```

As the example grows, group related checks into named procedures, just as we would group other related work:

```
def test_single_digits():
    check("0", 0.0)
    check("3", 3.0)
    check("7", 7.0)
    check("9", 9.0)

def main():
    test_single_digits()
    print("All tests passed.")
```

Later, main will also call `test_multiple_digits` and `test_decimal_numbers`. A test procedure does not run merely because we defined it. It needs a call. Keep the call to main at the bottom of the file. Run the tests after moving the repeated comparison into check. This is a small refactoring of test code. The failure message now includes the input as well as both values, so a failure is easier to investigate.

More than one digit

The next specification is that "42" produces 42.0. Add `check("42", 42.0)` to a new `test_multiple_digits` procedure and call it from main. Run it before changing the converter. The current function returns 4.0, because it reads only the first character.

Reading a whole number requires keeping the value so far. Each new digit moves the earlier digits one decimal place to the left:

Character read	Calculation	Value so far
"1"	$0 * 10 + 1$	1
"0"	$1 * 10 + 0$	10
"5"	$10 * 10 + 5$	105

```
def parse(text):
    value = 0.0
    for digit in text:
        value = value * 10 + ord(digit) - ord("0")
    return value
```

Run every test, including the single digits. Add cases such as "10", "105", and "007". The leading-zero case makes a decision visible: for this tool, leading zeros do not change the value.

Notice how little we need to hold in our heads. The change is a loop and an accumulation rule. Earlier behavior has tests we can rerun. If a single digit now fails, we have a small recent change to examine.

A decimal point

Next, specify a decimal example: add `check("12.5", 12.5)` in `test_decimal_numbers`, call that procedure from `main`, and run it. The current loop treats the decimal point as though it were a digit, producing the wrong answer. We need a different rule after the point.

Before the point, multiply the accumulated value by ten and add the next digit. After the point, add tenths, then hundredths, then thousandths.

```
def parse(text):
    value = 0.0
    place = 0.1
    after_point = False
    for digit in text:
        if digit == ".":
            after_point = True
        elif after_point:
            value += (ord(digit) - ord("0")) * place
            place /= 10
        else:
            value = value * 10 + ord(digit) - ord("0")
    return value
```

Keep "12.5" and add "0.25". Then try "12.34" and "10.02". The zero in "10.02" matters: it occupies the tenths place even though it contributes nothing to the value.

Comparing floating-point results

Our small whole numbers can be compared exactly. Decimal fractions need more care. Many decimal values have no exact binary floating-point representation, and arithmetic can introduce small rounding differences (3).

For these short examples, replace the equality check with `math.isclose`:

```
import math

def check(text, expected):
    actual = parse(text)
    if not math.isclose(actual, expected,
                        rel_tol=1e-12, abs_tol=1e-12):
        raise AssertionError(
            f"{text!r}: expected {expected!r}, got {actual!r}")
```

The relative tolerance allows a small difference in proportion to the values. The absolute tolerance provides a small allowance near zero (4). These tolerances are choices for this exercise, not universal settings for numerical software. A tolerance large enough to hide a misplaced decimal point would make a poor test.

We still need a known expected answer. Do not calculate it by copying the converter's algorithm into the test. Both copies could contain the same mistake.

A leading sign

Specify the meaning of a leading minus sign with `check("-12.5", -12.5)` in `test_signed_numbers`. Call it from `main` and run it. The decimal converter does not understand the minus sign. We can handle the sign before processing the digits:

```
sign = 1.0
if text.startswith("-", "+"):
    if text[0] == "-":
        sign = -1.0
    text = text[1:]
```

Place this code at the start of the function and change its last line to `return sign * value`. Run the tests. Add `"7"` and `"-0.25"`, keeping the unsigned examples too.

This implementation handles the optional plus sign while handling the minus sign. Small steps do not require awkward code that deliberately refuses a closely related case. The tests should still record the behavior we intend to keep.

What should happen to bad input?

So far we have concentrated on valid examples. The function still accepts things we have never agreed to support. For example, an empty string returns zero, and `"1.2.3"` is treated as though the second point did not matter.

We need a decision before we need code. For this draft, the accepted form is a sequence of ASCII digits, optionally preceded by one sign. One decimal point is allowed, with at least one digit somewhere in the number. Thus `".5"` and `"5."` are valid. Other strings raise `ValueError`.

Input	Decision for this version
"7", "-12.5", "+0.25"	Accept
"007"	Accept as 7.0
"" , "+", "-"	Reject: no digits
"12x", "1.2.3"	Reject: invalid form
".5", "5."	Accept as 0.5 and 5.0
" 7", "7 ", "1e3"	Reject for now

These decisions belong to our example. Another application might reasonably accept spaces or insist on digits on both sides of a decimal point. A test records the decision; it does not make the decision correct for every product.

Add a test procedure for rejection:

```
def check_rejected(text):
    try:
        parse(text)
    except ValueError:
        return
    raise AssertionError(f"{text!r}: expected ValueError")
```

Start with `check_rejected("")`. It fails because the converter returns zero. After removing an optional sign, check for an empty string and raise `ValueError` when it is empty. Run the suite. Next add `check_rejected("12x")` and watch that test fail. Add a digit check. Continue with the repeated-point and missing-digit cases, one at a time.

Once those checks work, they can be expressed together by splitting the unsigned text at the decimal point:

```
parts = text.split(".")
if (len(parts) > 2 or
    not any(parts) or
    any(c < "0" or c > "9" for part in parts for c in part)):
    raise ValueError("Expected digits with an optional decimal point")
```

Put this validation after removing the sign and before the accumulation loop. The explicit "0" through "9" comparison keeps the accepted digits within ASCII. Add acceptance checks for ".5" and "5." too. Keep all valid-input tests: a validator that rejects every string would pass all the rejection tests and still be useless.

The complete implementation and cumulative tests are in `code/06_invalid_input.py`. It is a teaching converter for short, ordinary values. Very long inputs, overflow, underflow, and correctly rounded conversion across the full floating-point range need further work. For application code, a language's established conversion routine is usually the sensible starting point. Here we are implementing the conversion to practice the development cycle.

What refactoring contributes

Getting a test to pass is not the end of the cycle. Read the working code. A repeated expression may deserve a name. An unclear variable may need a better one. Two checks may be doing the same work.

The extraction of check was a refactoring: the tests still asked the same questions, but their comparison and failure reporting lived in one place. Adding negative numbers was a feature change: it made a new promise. Keeping those changes separate helps us understand a failure.

A refactoring does not have to make the program more elaborate. Sometimes the clearest choice is to leave a short loop alone. Run the tests after any change, and remember that a green suite protects only the behavior it checks.

Specification first, one piece at a time

At each step, the test came before the behavior it specified. We decided what a second digit meant before adding the accumulation loop. We decided what a minus sign meant before adding sign handling. Tests for bad input made error behavior part of the specification too.

Writing the converter first and then adding tests can still give us useful checks. It misses this particular benefit: the need to decide what the next feature should do before becoming occupied with how to implement it.

We are not trying to resolve every question at the start. We are keeping the order clear within a small iteration. Specify a behavior. Implement it. Check it against the specification. Improve the code, then choose the next behavior.

A workable daily habit

When a test fails, read what it says before changing code. When a new test passes immediately, find out why. The behavior may already exist, or the test may not be exercising what we thought it was.

When a defect appears, turn the failing example into a test before repairing it. Keep that test. It gives us a way to notice the same defect returning.

This can make development more enjoyable. There is less need to wonder whether a small edit broke something familiar. A teammate has concrete examples to consult, and a failure points to a specific expectation. Tests cannot settle every requirements disagreement or prevent every difficult day. They can reduce a recurring source of uncertainty and rework.

Working through the example

The files in `code/` are checkpoints. Run each with `python3`, followed by its filename. `00_red.py` deliberately fails; the later files pass their cumulative tests. Read the checkpoints after trying each change yourself.

File	Behavior reached
<code>00_red.py</code>	First failing test for "7"
<code>01_first_pass.py</code>	Constant result passes that test
<code>02_single_digits.py</code>	Several different single digits
<code>03_whole_numbers.py</code>	Multiple digits and leading zeros
<code>04_decimals.py</code>	Decimal fractions and approximate comparison
<code>05_signs.py</code>	Optional leading sign
<code>06_invalid_input.py</code>	Rejection of unsupported string forms

A checkpoint is the result of several small decisions, not permission to write all its code before running a test. To reproduce a failure, add the next example to the previous checkpoint first.

For an extension, consider scientific notation. Begin with `"1e2"` giving `100.0`. Once that works, consider `"1E2"`, then a decimal mantissa such as `"12.34e2"`. Negative exponents introduce another decision. List possible examples, choose one, and run the existing tests after each change. Update the earlier rejection test for `"1e3"` deliberately when that requirement changes; do not leave contradictory expectations in the suite.

References

1. Martin Fowler. “Test Driven Development.” Updated December 11, 2023. <https://www.martinfowler.com/bliki/TestDrivenDevelopment.html>
2. Wikipedia contributors. “Test-driven development.” https://en.wikipedia.org/wiki/Test-driven_development
3. Python Software Foundation. “Floating-Point Arithmetic: Issues and Limitations.” Python 3 documentation. <https://docs.python.org/3/tutorial/floatingpoint.html>
4. Python Software Foundation. “math.isclose.” Python 3 documentation. <https://docs.python.org/3/library/math.html#math.isclose>

Questions for Thought

1. Before writing code for a leading minus sign, what decisions belong in its specification? Give a test that makes one of those decisions concrete.
2. A converter returns `7.0` for every input. Which test would you add first, and what would its failure tell you that the original test could not?
3. Your new test passes before you change the converter. How could you distinguish behavior that already works from a test that is not checking anything useful?
4. Why do we keep the single-digit tests after adding support for longer numbers? Give an example of a later change that could break an earlier case.
5. A caller asks us to accept spaces around a number. Who should decide, and what would you change in the tests before changing the implementation?
6. A teammate increases the comparison tolerance until a failing decimal test passes. What evidence would you want before accepting that change?
7. How could this way of working make a shared project less frustrating? What kinds of problems would still require conversation or other kinds of testing?
8. Why is `add(2, 3)` more informative than another equal positive pair after several equal-pair tests have passed?
9. After inspecting `add5`, we can write a test for its exceptional input. What does that test establish? What did the earlier passing suite establish?
10. Would you trust an addition implementation that passes a comprehensive suite but has not been inspected? How would the consequences of a wrong answer affect your decision?